

Table des matières

1) Présentation du projet et objectifs	3
Contexte de la SAÉ	3
Le jeu de bataille navale	3
Objectifs principaux	3
Mon rôle dans le projet	4
2) Environnement de développement et outils	5
Outils et technologies utilisés	5
Type de projet	5
Mise en place de l'environnement	5
3) Architecture de l'application	7
Modèle client / serveur	7
Communication en temps réel avec Socket.io	8
4) Développement et intégration multijoueur	9
Initialisation du serveur Node.js	9
Placement des navires et interface réactive	9
Gestion des tours et condition de victoire	10
Identification des joueurs et gestion des déconnexions	11
5) Méthodologie et organisation du travail	13
Phase 1 : Recherche et inspiration	13
Phase 2 : Développement et optimisation	13
Phase 3 : Intégration du multijoueur	13
6) Tests et validation	14
7) Défis et solutions	15
Défi 1 : Communication client / serveur	15
Défi 2 : Déconnexion d'un joueur	15
Défi 3 : Synchronisation et alternance des tours	15
Conclusion	16
Bilan	16
Compétences développées	16
Perspectives	16

Dans le cadre de la SAÉ 3.02 « **Développer des applications communicantes** », notre groupe devait concevoir une application capable d'échanger des données avec un serveur et d'interagir avec lui. Nous avons développé **un jeu multijoueur de bataille navale** en JavaScript : l'utilisateur place ses bateaux puis tente de toucher ceux de l'adversaire directement dans son navigateur, avec une communication en temps réel assurée par **Socket.io**. Le serveur a été développé avec **Node.js**. Ce projet a été réalisé **en groupe** : l'ensemble du design et du code a été créé collectivement par notre groupe avant que le projet ne soit poursuivi **individuellement**.

Donc pour cela voici l'infrastructure mise en place :

1) Présentation du projet et objectifs

Contexte de la SAÉ

La SAÉ 3.02 « **Développer des applications communicantes** » avait pour objectif de concevoir une application communicante avec gestion des requêtes, affichage dynamique de données et interaction avec le réseau et le serveur.

Le jeu de bataille navale

Le projet consiste en un **jeu multijoueur de bataille navale** développé en JavaScript. L'enjeu central : **synchroniser les actions de deux joueurs** via une architecture Node.js et Socket.io, garantissant une expérience de jeu fluide, interactive et sans latence perçue.



Fig. 1 – Aperçu du jeu Bataille Navale dans le navigateur

Cette capture montre le titre du jeu affiché dans le navigateur, point d'entrée de l'application accessible à tous les joueurs.

Problème à résoudre : Le projet visait à créer une application web permettant à deux joueurs de communiquer en temps réel. Il était nécessaire de gérer les échanges de données et de synchroniser l'affichage entre les participants pour garantir une expérience fluide et cohérente.

Objectifs principaux

L'objectif principal était de développer un jeu web multijoueur fonctionnel, capable d'utiliser les **sockets** pour la communication et de respecter un cahier des charges technique précis :

- Implémenter des WebSockets pour une **transmission instantanée** des tirs et placements
- Gérer les événements serveur pour synchroniser l'état de la partie entre les clients
- Offrir une interface réactive permettant un **placement de navires intuitif** et un retour visuel immédiat
- Assurer une communication bidirectionnelle efficace réduisant les conflits de données en jeu
- Gérer de manière autonome les **tours de jeu** et les **conditions de victoire** côté serveur
- Sécuriser les parties grâce à un système d'**identification des joueurs** par sessions
- Développer un back-end robuste sous Node.js pour centraliser la logique de jeu
- Concevoir un système de gestion des **déconnexions** pour maintenir l'intégrité des sessions

Mon rôle dans le projet

Dans ce projet **réalisé en groupe**, j'ai principalement travaillé sur le **développement du code** et le **design**, tout en contribuant, dans une moindre mesure, à la partie **back-end**. Mon rôle a inclus l'implémentation des fonctionnalités du jeu et l'optimisation de l'expérience utilisateur.

2) Environnement de développement et outils

Outils et technologies utilisés

Le projet a été développé et testé dans un environnement virtualisé, puis déployé sur un serveur Web :

Node.js	Serveur d'application : hébergement du code serveur et gestion des connexions
JavaScript	Langage utilisé côté client (navigateur) et côté serveur
Socket.io	Bibliothèque de communication temps réel (WebSockets)
HTML / CSS	Structure et design de l'interface du jeu
PHP / MySQL (MariaDB)	Technologies de gestion des requêtes et des données étudiées dans la SAÉ
Debian	Système d'exploitation du serveur d'hébergement
VirtualBox	Virtualisation de l'environnement de test et de déploiement

Tableau 1 – Outils et technologies utilisés pour le projet

Type de projet

Ce projet est un projet **personnel et d'étude**. Il s'inscrit dans le cadre pédagogique de la SAÉ 3.02 du BUT Réseaux & Télécoms.

Mise en place de l'environnement

Le projet étant basé sur Node.js, nous commençons par vérifier la présence du moteur JavaScript et de son gestionnaire de paquets :

```
node --version  
npm --version
```

La sortie confirme la version de Node.js installée (Node.js LTS) et la présence de **npm**, nécessaire à l'installation de Socket.io et des dépendances du projet :

```
npm init -y  
npm install express socket.io
```

Ces commandes initialisent le fichier `package.json` et installent Express, le framework HTTP, ainsi que Socket.io, la bibliothèque de communication temps réel.

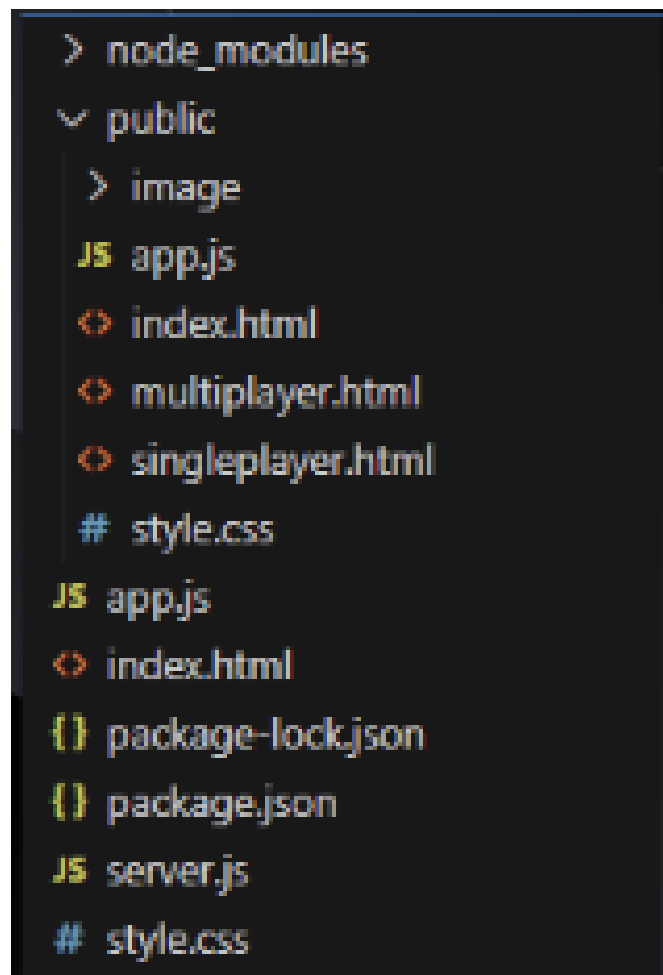


Fig. 2 – Arborescence du projet Bataille Navale

Cette arborescence organise le projet en séparant le serveur (`server.js`), les pages du jeu (`index.html`, `multiplayer.html`, `singleplayer.html`) et les dépendances installées (`node_modules`), garantissant une maintenance et des déploiements plus simples.

3) Architecture de l'application

Modèle client / serveur

L'application repose sur une architecture **client / serveur** classique. Le serveur Node.js centralise la logique de jeu et agit comme **autorité unique** : c'est lui qui valide les tirs, alterne les tours et détecte la victoire. Les clients (navigateurs) n'affichent que l'état qui leur est transmis, ce qui évite les conflits de données.

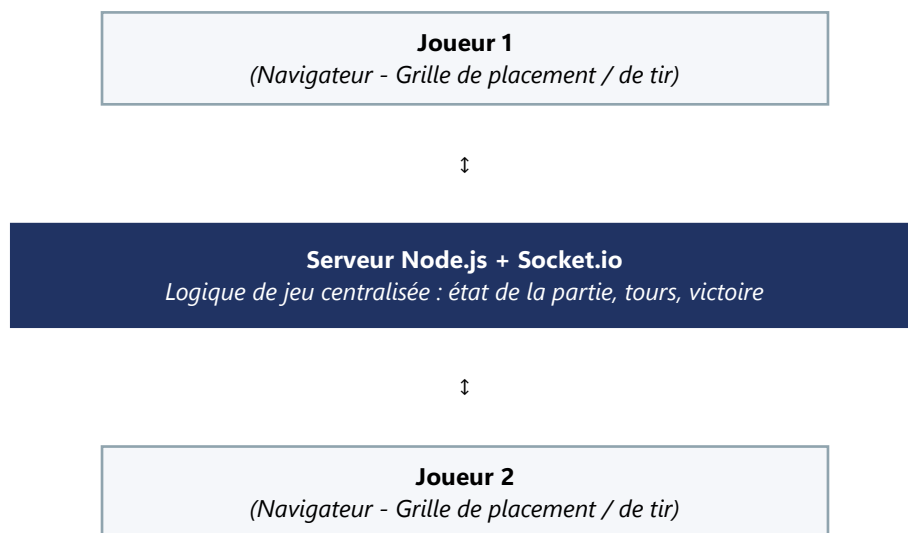


Fig. 3 – Architecture de l'application Bataille Navale

La mise en œuvre de cette architecture s'appuie sur le fichier `server.js`, qui instancie le serveur HTTP, lui branche Socket.io et met à disposition le contenu statique du jeu :

```
const express = require('express')
const path = require('path')
const http = require('http')
const PORT = process.env.PORT || 8080
const socketio = require('socket.io')
const app = express()
const server = http.createServer(app)
const io = socketio(server)

// Définir le dossier
app.use(express.static(path.join(__dirname, "public")))

// Lancer le serveur
server.listen(PORT, () => console.log(`Le serveur est lancé sur http://localhost:${PORT}`))
```

Fig. 4 – Code serveur Node.js : initialisation d'Express, HTTP et Socket.io (`server.js`)

Cette capture confirme que le serveur écoute sur le port défini par la variable d'environnement `PORT` et annonce son lancement dans la console : le canal de communication temps réel est prêt à recevoir les joueurs.

Communication en temps réel avec Socket.io

Socket.io s'appuie sur le protocole **WebSocket** pour établir un canal de communication bidirectionnel et persistant entre le navigateur et le serveur. Contrairement à une requête HTTP classique qui doit être renouvelée à chaque échange, la connexion WebSocket reste ouverte et permet d'émettre des événements instantanément dans les deux sens.

Le serveur conserve un **modèle de données centralisé** (grilles, navires, tour courant). À chaque action d'un joueur, il met à jour cet état puis diffuse les changements aux clients intéressés. Ce fonctionnement garantit qu'aucun joueur ne peut altérer l'état de la partie depuis son navigateur.

<code>connection</code>	Établit la connexion d'un joueur au serveur	→ Serveur
<code>placement</code>	Envoie la position des navires du joueur	Client → Serveur
<code>tir</code>	Transmet un tir avec ses coordonnées	Client → Serveur
<code>resultat-tir</code>	Diffuse le résultat (touché / raté / coulé)	Serveur → Client
<code>tour</code>	Informe les joueurs de qui doit jouer	Serveur → Client
<code>victoire</code>	Signale la fin de partie et le vainqueur	Serveur → Client
<code>deconnexion</code>	Notifie la perte de connexion d'un joueur	Client → Serveur

Tableau 2 – Principaux événements Socket.io du jeu

4) Développement et intégration multijoueur

Initialisation du serveur Node.js

Le serveur est initialisé avec Express et Socket.io. À chaque nouvelle connexion, un identifiant de session est attribué au joueur, ce qui permet de le reconnaître et de l'associer à sa partie :

```
const express = require("express");
const http = require("http");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server);

let partie = {
  joueurs: [],      // joueurs connectés avec leur session
  grilles: {},      // positions des navires de chaque joueur
  tour: null,       // joueur dont c'est le tour
  terminee: false,
};

io.on("connection", (socket) => {
  socket.emit("connexion", { id: socket.id });
});
```

Cette étape confirme que le serveur écoute bien les connexions entrantes et qu'un canal temps réel est ouvert avec chaque joueur.

Placement des navires et interface réactive

L'interface permet un **placement intuitif** des navires : le joueur glisse et oriente ses bateaux sur sa grille avant que la partie ne commence. Une fois le placement validé, l'événement `placement` est transmis au serveur, qui enregistre les coordonnées des navires dans son modèle de données.

Un **retour visuel immédiat** est ensuite affiché pour chaque tir (raté, touché, coulé), ce qui rend le jeu fluide et agréable sans rechargement de la page.

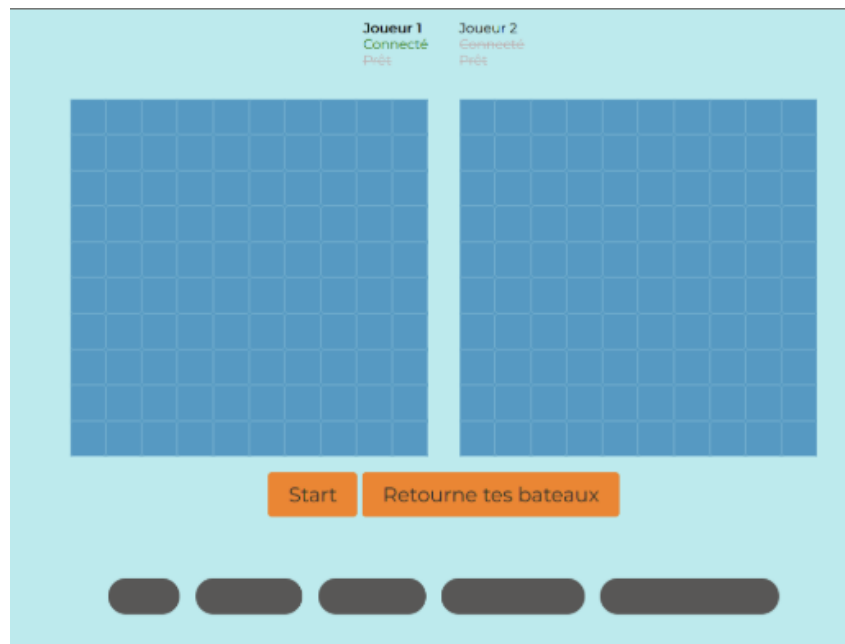


Fig. 5 – Interface de placement des navires sur la grille de jeu

Cette capture montre la grille de placement : le joueur dispose ses bateaux à l'emplacement souhaité avant le début de la partie, le serveur enregistrant ensuite ces coordonnées via l'événement `placement`.

Gestion des tours et condition de victoire

La logique de jeu est centralisée sur le serveur : gestion des tours, validation des tirs et détection des conditions de victoire. Cette centralisation évite tout conflit de données entre les deux clients.

```
socket.on("tir", (data) => {
  if (partie.tour !== socket.id || partie.terminee) return;

  const adversaire = partie.grilles[adversaireOf(socket.id)];
  const touche = adversaire.cases[data.x][data.y];

  socket.emit("resultat-tir", { x: data.x, y: data.y, touche });
  adversaireOf(socket).emit("resultat-tir", { x: data.x, y: data.y, touche });

  if (flotteCoulee(adversaire)) {
    io.emit("victoire", { vainqueur: socket.id });
  } else {
    changerDeTour();
  }
});
```

Le serveur vérifie systématiquement que l'action provient bien du joueur dont c'est le tour avant de la traiter, puis transmet l'état synchronisé aux deux clients. La condition de victoire

est détectée côté serveur : dès que toute la flotte d'un joueur est coulée, l'événement victoire est diffusé à tous.

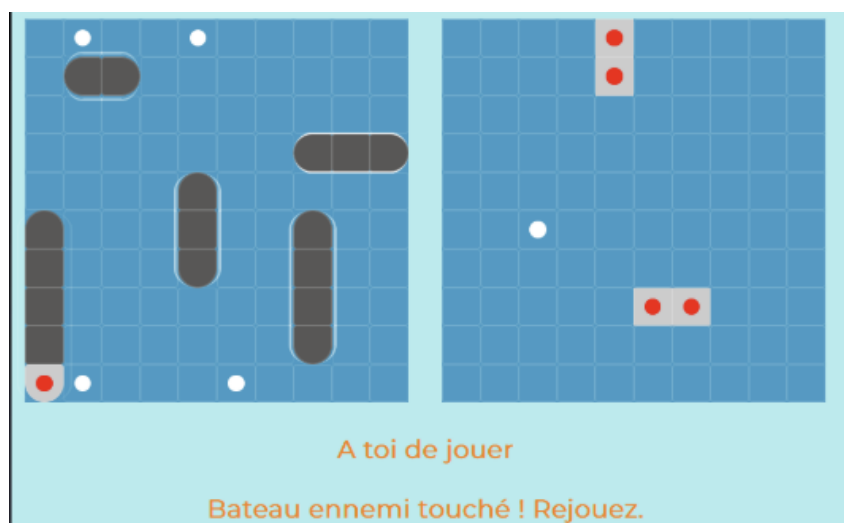


Fig. 6 – Partie en cours : indicateur de tour et résultat d'un tir

Cette capture illustre le système de tours et le retour visuel du jeu : le message « **A toi de jouer** » indique au joueur actif que c'est son tour, tandis que « **Bateau ennemi touché ! Rejouez.** » confirme que le tir a touché un navire adverse et que le joueur peut rejouer. Ces informations sont diffusées en temps réel par le serveur aux deux clients.

Identification des joueurs et gestion des déconnexions

Un point délicat du projet concerne la **stabilité de la session** : chaque joueur est identifié par un identifiant de session unique. En cas de déconnexion, le serveur signale l'événement à l'adversaire, termine proprement la partie et libère les ressources :

```
socket.on("disconnect", () => {  
  if (partie.terminee) return;  
  
  io.emit("adversaire-deconnecte", { id: socket.id });  
  partie.terminee = true;  
});
```

Ce mécanisme garantit qu'une partie ne reste pas bloquée après la perte de connexion d'un joueur, et prévoit de reprendre la partie si le joueur se reconnecte.

Côté client, l'attribution du numéro de joueur passe par l'événement `player-number` : le serveur renvoie la place du joueur tant que la partie n'est pas complète, ou signale qu'elle est pleine :

```
// Multijoueur
function startMultiPlayer() {
  const socket = io();

  // Obtenir le numero de joueur
  socket.on('player-number', num => {
    if (num === -1) {
      infoDisplay.innerHTML = "Désoler le serveur est plein"
    } else {
      playerNum = parseInt(num)
      if(playerNum === 1) currentPlayer = "ennemi"

      console.log(playerNum)

      // Obtenir le statut d'un autre joueur
      socket.emit('check-players')
    }
  })
}
```

Fig. 7 – Code client du mode multijoueur : connexion Socket.io et obtention du numéro de joueur (app.js)

Cette capture confirme le lien entre le client et le serveur : `socket.emit("check-players")` interroge l'état la partie au démarrage, et l'événement `player-number` attribue à chaque joueur son identifiant de session au sein de la partie.

5) Méthodologie et organisation du travail

Le projet a été mené en trois phases principales, allant de la recherche d'inspiration jusqu'au déploiement final.

Phase 1 : Recherche et inspiration

Lors de cette première phase, nous avons :

- Analysé des projets de bataille navale existants sur le web afin d'en comprendre les mécanismes de jeu et les choix d'interface
- Étudié et adapté des codes existants pour répondre aux besoins spécifiques du projet
- Amélioré le design afin d'optimiser l'expérience utilisateur, en travaillant sur l'ergonomie des grilles et la clarté des retours visuels

Phase 2 : Développement et optimisation

La seconde phase a consisté à structurer et fiabiliser le jeu :

- Conversion du code initial vers **Node.js** pour la partie serveur
- Mise en place du projet sur le serveur et configuration initiale de l'environnement
- Optimisation du code et correction des erreurs pour garantir la stabilité de l'application

Phase 3 : Intégration du multijoueur

Enfin, la dernière phase a porté sur la communication temps réel :

- Mise en place du **mode multijoueur**
- Intégration de **Socket.io** pour assurer la communication en temps réel entre les clients et le serveur
- Tests, débogage et optimisation spécifiques aux interactions multijoueur (synchronisation des grilles, alternance des tours, détection de victoire)

6) Tests et validation

Les tests ont été réalisés avec **deux navigateurs** connectés simultanément au serveur, permettant de vérifier chaque fonctionnalité du jeu :

Synchronisation du placement des navires entre les deux joueurs	Validé
Transmission instantanée des tirs et affichage des résultats de chaque côté	Validé
Alternance stricte des tours (aucun joueur ne joue hors de son tour)	Validé
Détection de la victoire et fin de partie propre	Validé
Comportement en cas de déconnexion volontaire d'un joueur	Validé

Tableau 3 – Synthèse des tests multijoueur

Les tests ont conduit à plusieurs corrections, notamment sur l'ordre des événements lors d'un tir (validation avant diffusion) et sur le nettoyage de l'état de la partie en fin de session.

7) Défis et solutions

Défi 1 : Communication client / serveur

Défi : Implémenter correctement la communication client/serveur avec les protocoles adaptés et assurer la gestion des données.

Solution : Utiliser Socket.io pour émettre des événements en temps réel à chaque action, et maintenir un modèle de données centralisé côté serveur pour diffuser les mises à jour aux clients. Le serveur devient la seule source de vérité de l'état de la partie, chaque client ne recevant que les informations qui le concernent.

Défi 2 : Déconnexion d'un joueur

Défi : Que se passe-t-il si un joueur se déconnecte ou perd la connexion ? Comment gérer une reconnexion ou la fin de partie ?

Solution : Implémenter un système d'identification des joueurs via des sessions ou des tokens, et prévoir des mécanismes pour reprendre la partie ou terminer proprement en cas de déconnexion. Le serveur détecte l'événement `disconnect`, en informe l'adversaire et finalise la partie de manière cohérente.

Défi 3 : Synchronisation et alternance des tours

Défi : Synchroniser l'affichage entre les deux joueurs et alterner les tours sans conflit de données.

Solution : La gestion autonome des tours de jeu et des conditions de victoire est effectuée **côté serveur**. Le serveur vérifie systématiquement que l'action provient bien du joueur dont c'est le tour avant de la traiter, puis transmet l'état synchronisé aux deux clients.

Conclusion

Bilan

La SAÉ 3.02 a pleinement atteint ses objectifs : nous avons conçu une application communicante fonctionnelle, jouable à deux navigateurs en temps réel. Le serveur Node.js associé à Socket.io assure une communication bidirectionnelle fluide, et la centralisation de la logique de jeu côté serveur garantit la cohérence des parties.

Ce projet nous a permis de répondre aux exigences du cahier des charges : gestion des requêtes, affichage dynamique des données et interaction réelle entre l'application et le serveur.

Compétences développées

Les compétences suivantes ont été développées ou renforcées :

- Développement d'applications Web **communicantes** avec Node.js
- Utilisation des **WebSockets** et de Socket.io pour le temps réel
- Centralisation et synchronisation d'un état de jeu partagé
- Conception d'interfaces réactives en JavaScript
- Identification des joueurs par sessions et gestion des déconnexions
- Tests et débogage d'interactions temps réel

Perspectives

Le jeu constitue une base solide pouvant être enrichie :

- Gestion de **parties multiples** simultanées (salles, matchmaking)
- Système de **reconnexion** complet avec restauration de l'état de la partie
- Ajout d'un **classement** et de statistiques de jeu
- Durcissement de la sécurité (validation renforcée des entrées, chiffrement des échanges via HTTPS)
- Migration des données vers la base de données **MariaDB** pour la gestion des comptes joueurs